

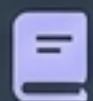
Programming in Plain English

Mastering ChatGPT & Prompt
Engineering Foundations

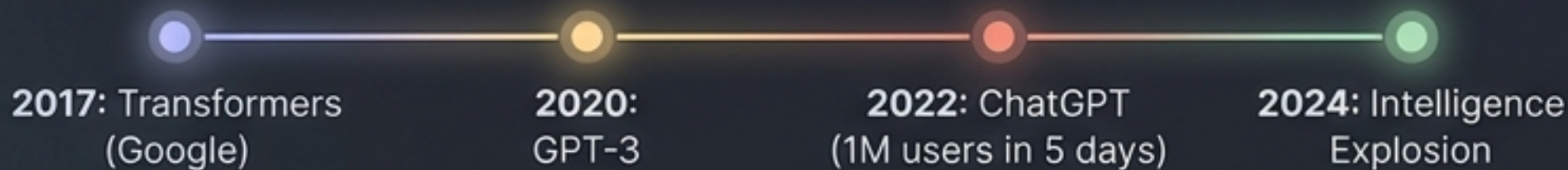
Neurospire AI | MSME Registered Edtech

```
function generateGreeting(name) {  
  const greeting = 'Hello, ';  
  const message = `${greeting}\${name}!  
Welcome to the code.`;  
  console.log(message);  
  return message;  
}
```

Language models do not think—they predict the highest probability outcome.



The Training: Trained on billions of words from the internet, books, and code. Like a student who has read every book in a library.



The existential productivity multiplier.

Task	Without AI	With AI
Write an email	20 minutes	2 minutes
Debug code	1 hour searching	5 minutes
Research a topic	Read 10 articles	Summarized in 30 sec
Build a landing page	2 days	2 hours

"AI won't replace you. A person using AI will."

The anatomy of a perfectly engineered prompt.

✗ Vague

> fix my code

```
function addNumbers(a, b) {  
  return a + b;  
}  
  
// Example usage:  
console.log(addNumbers(5, 10)); // Output: 15  
console.log(addNumbers("5", "10")); // Output: 510  
console.log(addNumbers("hello", 10)); // Output: hello10
```

✓ Specific

I'm learning JavaScript. This function is supposed to add two numbers but it returns NaN. } [Context]

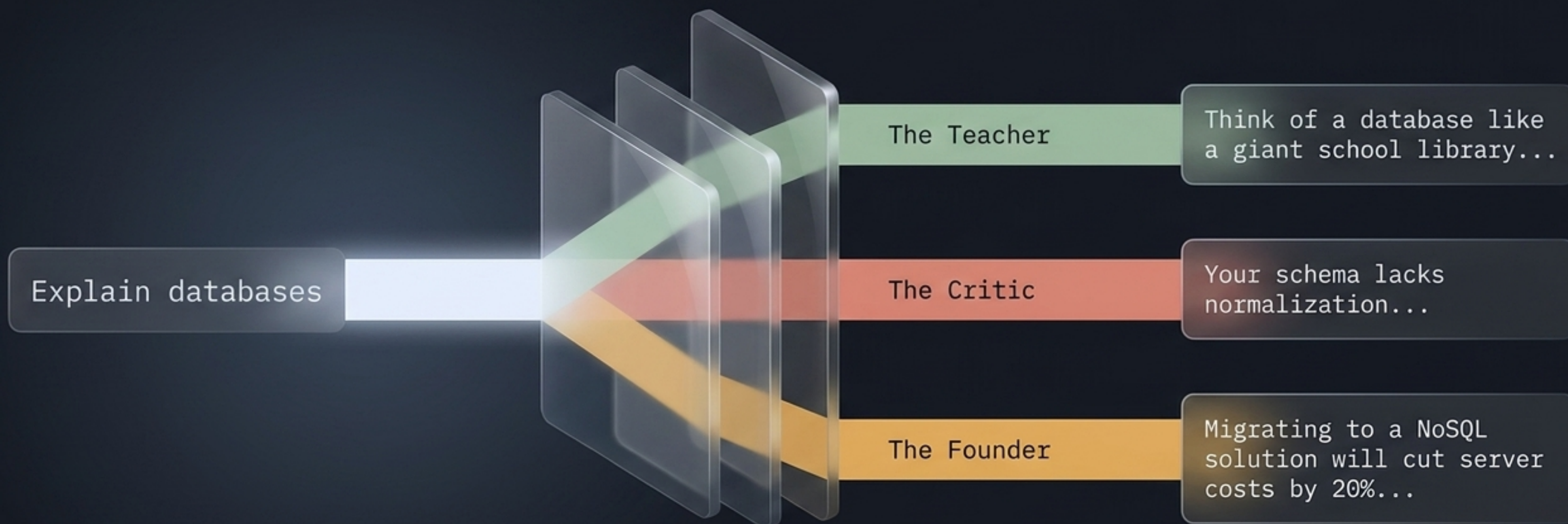
Here's the code: [paste]. } [Specific Input]

Find the bug and explain why it happened. } [Action/Role]

[Format & Length]

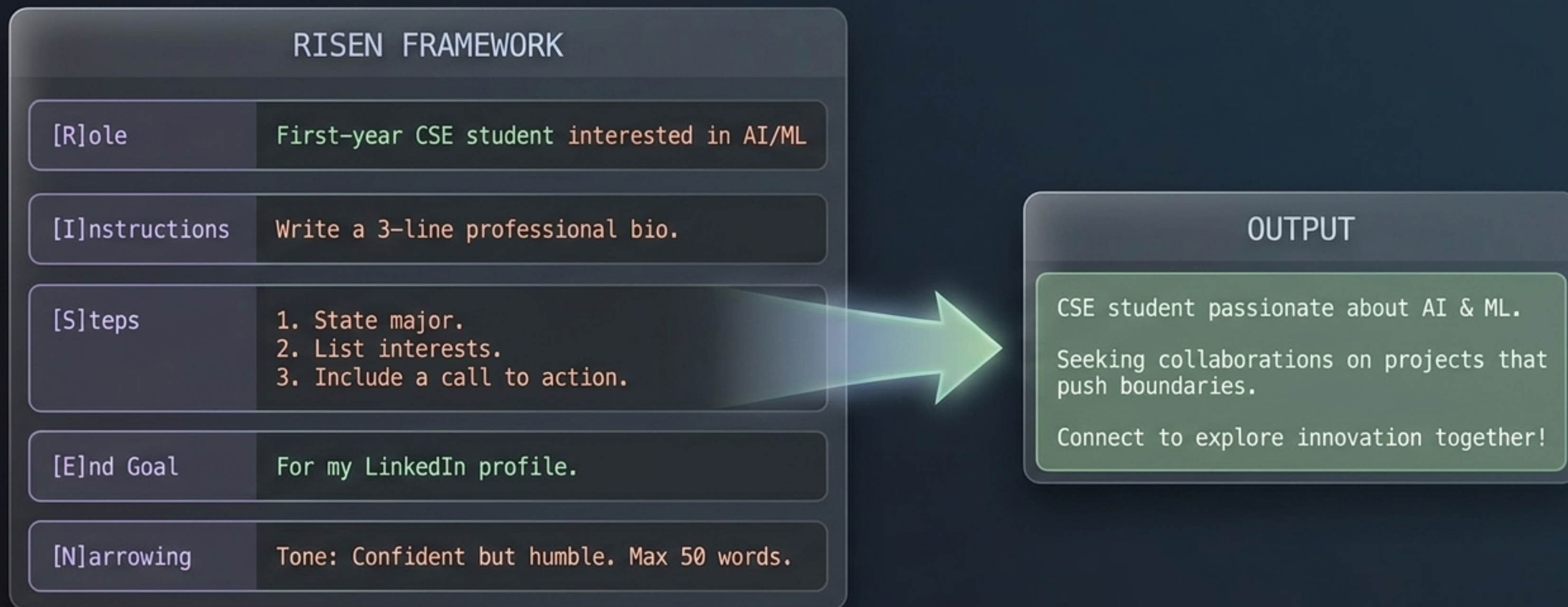
```
// [Specific Output]  
function addNumbers(a, b) {  
  // Convert inputs to numbers to prevent string concatenation or NaN issues  
  const numA = Number(a);  
  const numB = Number(b);  
  
  // Check if inputs are valid numbers  
  if (isNaN(numA) || isNaN(numB)) {  
    return "Error: Inputs must be valid numbers.";  
  }  
  
  // Return the sum  
  return numA + numB;  
}  
  
// Example usage:  
// console.log(addNumbers(5, 10)); // Output: 15  
// console.log(addNumbers("5", "10")); // Output: 15  
// console.log(addNumbers("hello", 10)); // Output: Error: Inputs must be valid numbers.
```

Constrain the output by applying a cognitive lens.



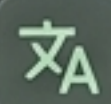
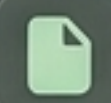
[!] Pro Tip: 'Act as an expert [X]' beats 'Help me with [Y]'.
The more specific the role, the better the output.

Eliminate hallucination through structured frameworks.



The dual engine: AI for technical execution and deep research.

Coding Console

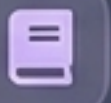
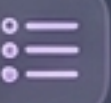
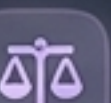
-  Explain this code
-  Debug this error
-  Instant starter code
-  Convert Python to JS
-  Auto-generate documentation

> User: "I have a JS array of student names. Write a function that sorts them alphabetically and removes duplicates."



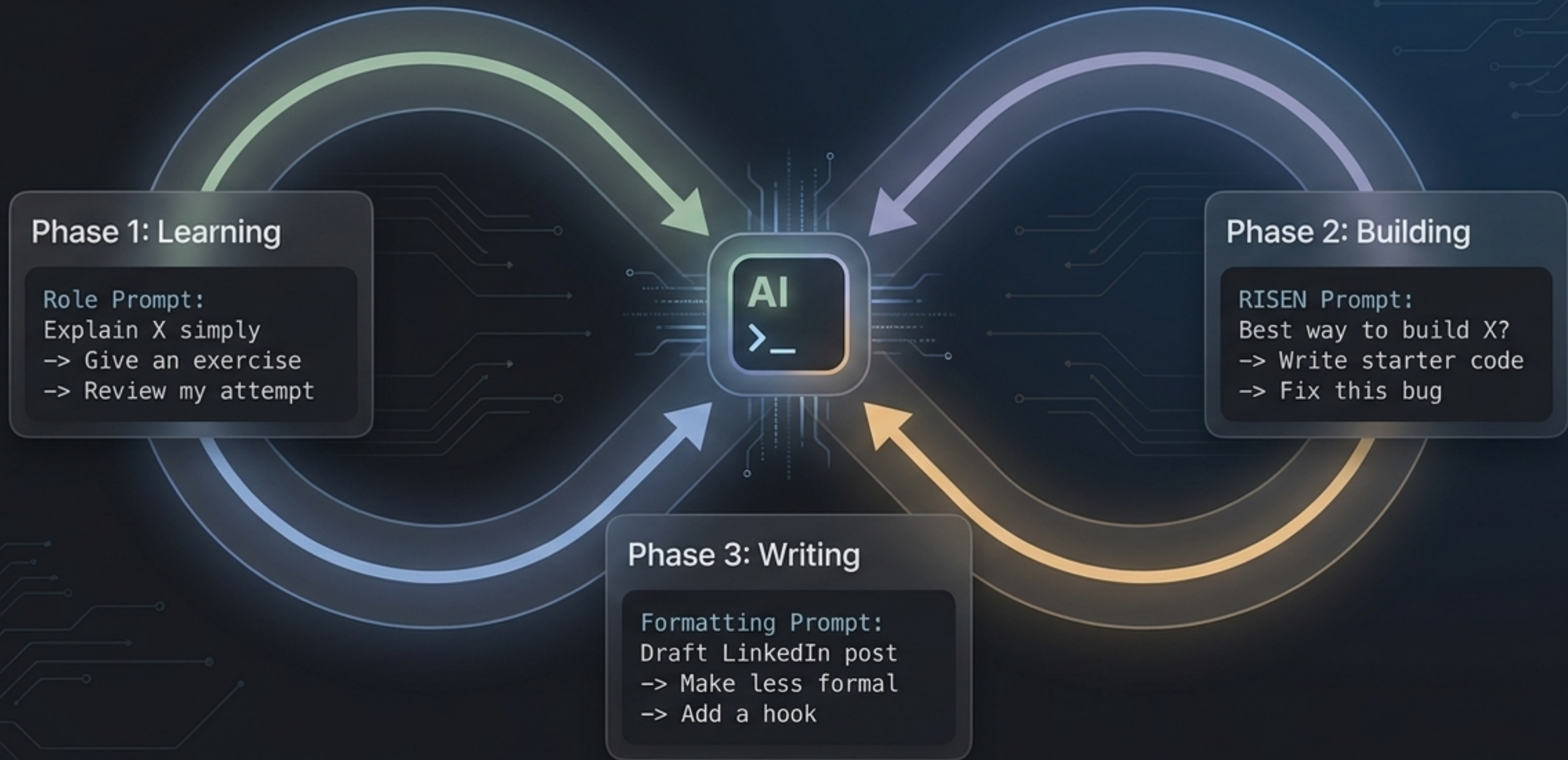
```
const uniqueSortedNames = (names) =>
  [...new Set(names)].sort();
```

Research Terminal

-  Summarize topics for beginners
-  5 Pros / 5 Cons of [Tech]
-  Identify common beginner mistakes
-  Compare React vs Vue

SYSTEM WARNING: AI can hallucinate. Never blindly trust. Always verify code by executing it.

The AI-First Developer Loop



System Initialization: The 5 Rules of AI Mastery.



- 01 **Recognize the Engine:** It is massive-scale pattern recognition, not magic.
- 02 **Provide the Context:** A prompt without context is a guaranteed failure.
- 03 **Apply the Lens:** Use Role Prompting to constrain and focus the output structure.
- 04 **Use the Framework:** Deploy RISEN to turn casual chats into engineered outputs.
- 05 **Trust but Verify:** Always run the code. Always own the final output.

> STATUS: PROFICIENT

Next Module Initialization...

Week 1 – Topic 2: Cursor AI Basics

AI-assisted coding — writing code at the speed of thought.



Neurospire AI

Innovating Education. Registered with MSME.

